Machine Learning — Basics to Expert

12 core concepts with Python examples · from first principles to production

scikit-learn . TensorFlow . PyTorch . Keras . XGBoost . May 2025

Topic 1 — What is Machine Learning?

Beginner

What is Machine Learning?

Traditional programming: you write explicit rules. Machine learning: you give the machine examples and it learns the rules itself.

> *"Instead of writing if has fins AND breathes underwater then fish, you show 10,000 pictures and the model figures out what makes a fish."*

Three types of ML

Type	What it needs	What it learns	Example
Supervised	Labelled examples (input + answer)	Mapping from inputs to outputs	Spam detection, price prediction
Unsupervised	Unlabelled data only	Hidden patterns and structure	Customer segmentation, anomaly detection
Reinforcement	Rewards and penalties	Actions that maximise reward	Game-playing AI, robotics

The ML pipeline

```
Collect data -> Clean data -> Choose model -> Train -> Evaluate -> Tune -> Deploy

# Data scientists spend 60-80% of their time on data collection and cleaning
# Garbage data -> garbage model, regardless of algorithm sophistication
```

Topic 2 — Supervised Learning

Beginner

Supervised Learning

You provide input-output pairs (training data). The model learns a mapping function f(x) = y so it can predict outputs for new, unseen inputs.

> *"Like a student studying past exam papers with answer keys. After seeing 1,000 examples, they can answer new questions."*

Regression	Classification
Output is a continuous number	Output is a discrete category
House price, temperature, salary	Spam/not spam, cat/dog, disease/healthy

```
from sklearn.linear_model import LinearRegression

X_train = [[800],[1000],[1200],[1500],[2000]]
y_train  = [160000,200000,240000,300000,400000]

model = LinearRegression()
model.fit(X_train, y_train)

model.predict([[1300]])
# Output: [260000]  — model learned: price ~ 200 x sqft
```

Topic 3 — Unsupervised Learning

Intermediate

Unsupervised Learning

You only give inputs — no correct answers. The model discovers structure, groupings, or relationships on its own.

> *"Like sorting foreign coins into groups by size and colour — no one told you the categories."*

Four main techniques

- **Clustering** — group similar items. Customer segmentation, document grouping.
- **Dimensionality reduction** — compress features. PCA, t-SNE for visualisation.
- **Anomaly detection** — learn normal, flag outliers. Fraud detection.
- **Association rules** — "people who buy X also buy Y." Market basket analysis.

```
from sklearn.cluster import KMeans
import numpy as np

customers = np.array([
    [10,20],[12,18],[9,22],    # frequent, low spend
    [80, 2],[90, 1],[75, 3],   # rare, high spend
    [40, 8],[45, 9],[38,10],   # moderate
])

kmeans = KMeans(n_clusters=3, random_state=42)
kmeans.fit(customers)
print(kmeans.labels_)
# [0,0,0, 1,1,1, 2,2,2]  — 3 groups found automatically
```

Topic 4 — Regression

Beginner

Regression — Predict a Continuous Number

Regression finds the mathematical relationship between input features and a continuous output value.

Algorithm	When to use
Linear regression	When relationship is approximately linear
Polynomial regression	When the data follows a curve
Ridge / Lasso	Linear + regularisation to prevent overfitting
Logistic regression	Despite the name, this is for classification — outputs probability

```
# Measuring error — RMSE (Root Mean Squared Error)
predictions = [260000, 310000, 195000]
actuals     = [265000, 300000, 210000]

mse  = sum((p-a)**2 for p,a in zip(predictions,actuals)) / len(actuals)
rmse = mse ** 0.5
# RMSE ~ 10,800 — average prediction error in pounds
```

Topic 5 — Classification

Beginner

Classification — Predict a Category

Given input features, predict which category the input belongs to. Binary: two classes. Multi-class: many classes.

Logistic regression for classification

Despite the name, logistic regression is a classifier. It outputs a probability via the sigmoid function, then applies a threshold.

```
from sklearn.linear_model import LogisticRegression

# Email features: [word_count, exclamation_marks, has_link, caps_ratio]
X_train = [[120,1,0,0.05],[30,5,1,0.40],[200,0,0,0.02],[15,8,1,0.60]]
y_train  = [0, 1, 0, 1]   # 0=not spam, 1=spam

clf = LogisticRegression()
clf.fit(X_train, y_train)

clf.predict([[50,6,1,0.45]])           # [1]  spam!
clf.predict_proba([[50,6,1,0.45]])     # [0.15, 0.85]  85% spam probability
```

> *Precision matters for spam filters (false alarms annoying). Recall matters for cancer detection (missing a case is dangerous). Choose your metric based on which error costs more.*

Topic 6 — Decision Trees and Ensemble Methods Intermediate

Decision Trees and Ensemble Methods

A decision tree splits data at each node based on the feature that best separates the classes, building a flowchart of yes/no questions.

> *"Like a flowchart a doctor uses: does the patient have a fever? Yes: does the patient have a cough? Yes: likely flu."*

Example — loan approval tree

```
Salary > 30,000?
    |-- Yes --> Employed > 2 years?
    |           |-- Yes --> APPROVE
    |           |-- No  --> Credit score > 650?
    |                       |-- Yes --> APPROVE
    |                       |-- No  --> REJECT
    |-- No  --> REJECT
```

Ensemble methods — combine many trees

Method	How it works	Strength
Random Forest	100+ trees on random subsets, majority vote	Robust, avoids overfitting, fast to train
XGBoost	Trees built sequentially, each fixes previous errors	State of art for structured data, Kaggle winner
AdaBoost	Weight misclassified samples more in each round	Good with weak learners

```
from sklearn.ensemble import RandomForestClassifier

X = [[35000,3,720],[28000,1,580],[55000,8,800],[22000,0,500]]
y = [1,0,1,0]  # 1=approve, 0=reject

rf = RandomForestClassifier(n_estimators=100, random_state=42)
rf.fit(X, y)

for name, imp in zip(['salary','years','credit'], rf.feature_importances_):
    print(f"{name}: {imp:.2f}")
# salary: 0.42   years: 0.31   credit: 0.27
```

Topic 7 — Clustering (K-Means)

Intermediate

Clustering — Group Similar Things Together

K-Means assigns each point to the nearest centroid, then recomputes centroids, repeating until stable.

Step by step

Step 1: Initialise — place K centroids randomly in the data space

Step 2: Assign — assign each point to its nearest centroid (Euclidean distance)

Step 3: Move — recalculate each centroid as the mean of all assigned points

Step 4: Repeat — repeat until centroids stop moving (convergence)

```
from sklearn.cluster import KMeans

# Elbow method -- choose optimal K
inertias = []
for k in range(1, 11):
    km = KMeans(n_clusters=k, random_state=42)
    km.fit(X)
    inertias.append(km.inertia_)
# Plot K vs inertia -- pick the "elbow" (bend in the curve)

# Fit 3 clusters
kmeans = KMeans(n_clusters=3, random_state=0).fit(X)
print("Labels:", kmeans.labels_)
print("Centres:", kmeans.cluster_centers_)
```

Topic 8 — Neural Networks

Advanced

Neural Networks

Layers of interconnected nodes where each connection has a weight. Training adjusts weights so the network makes better predictions.

> *"Like a chain of filters — early layers detect edges, later layers detect shapes, final layers detect faces."*

Activation functions

Function	Formula	Used for
ReLU	max(0, x)	Hidden layers — most common
Sigmoid	1 / (1 + e^-x)	Binary output layer (0 to 1)
Softmax	e^xi / sum(e^x)	Multi-class output — probabilities sum to 1
Tanh	(e^x - e^-x) / (e^x + e^-x)	Recurrent networks (-1 to 1)

```
import tensorflow as tf

model = tf.keras.Sequential([
    tf.keras.layers.Dense(64, activation='relu', input_shape=(10,)),
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dropout(0.3),           # prevents overfitting
    tf.keras.layers.Dense(1, activation='sigmoid'),
])

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.fit(X_train, y_train, epochs=50, batch_size=32, validation_split=0.2)
```

Topic 9 — Deep Learning

Expert

Deep Learning — CNNs, RNNs, Transformers

Architecture	Input type	Key idea	Real-world use
CNN	Images	Convolutional filters detect local features	Facial recognition, medical imaging, self-driving cars
RNN / LSTM	Sequences	Hidden state carries memory forward	Speech recognition, translation, time series
Transformer	Any tokens	Self-attention: every token attends to every other	GPT, Claude, BERT, Whisper, Stable Diffusion
GAN	Random noise	Generator vs discriminator adversarial training	Deepfakes, image synthesis, data augmentation

```
# CNN for handwritten digit recognition (MNIST)
model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(32,(3,3),activation='relu',input_shape=(28,28,1)),
    tf.keras.layers.MaxPooling2D(2,2),
    tf.keras.layers.Conv2D(64,(3,3),activation='relu'),
    tf.keras.layers.MaxPooling2D(2,2),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10,  activation='softmax'),  # 10 digit classes
])
# Achieves ~99.2% accuracy on handwritten digits
```

> *"Attention is All You Need" (2017) introduced the Transformer architecture. All major LLMs — GPT, Claude, Gemini, LLaMA — are based on it. Transformers are fully parallelisable unlike RNNs, enabling training at massive scale.*

Topic 10 — Model Evaluation

Intermediate

Model Evaluation — Is the Model Actually Good?

A model that always predicts "no cancer" achieves 99% accuracy on a 1% cancer dataset — but catches zero cases. Use the right metric for your problem.

The confusion matrix

```
                Predicted +      Predicted -
Actual +    True Positive (TP)  False Negative (FN)   <- missed cases
Actual -   False Positive (FP)   True Negative (TN)   <- false alarms

Precision = TP / (TP + FP)   -- of predicted positives, how many correct?
Recall    = TP / (TP + FN)   -- of actual positives, how many caught?
F1 Score  = 2 * P * R / (P + R)   -- harmonic mean
ROC-AUC   -- overall quality across all thresholds
```

Situation	Best metric	Reason
Spam filter	Precision	False alarms (ham in spam folder) are annoying
Cancer detection	Recall	Missing a case (false negative) is dangerous
Balanced classes	F1 score	Equally penalises precision and recall errors
Class imbalance	ROC-AUC	Robust to skewed class distributions

Topic 11 — Overfitting and Underfitting

Intermediate

Overfitting and Underfitting

Overfitting: model memorises training data but fails on new data. Underfitting: model is too simple to learn the pattern. You want the sweet spot.

> *"Overfitting is a student who memorises exam questions word-for-word but fails a rephrased version. Underfitting is the student who did not study at all."*

Problem	Train error	Test error	Fix
Underfitting	High	High	More complex model, more epochs, feature engineering

Problem	Train error	Test error	Fix
Good fit	Low	Low (similar)	Deploy it — this is what you want
Overfitting	Very low	High	More data, regularisation, dropout, early stopping

```
# Solutions for overfitting
# 1. L2 Regularisation (Ridge)
from sklearn.linear_model import Ridge
model = Ridge(alpha=1.0)   # alpha controls strength

# 2. Dropout in neural networks
tf.keras.layers.Dropout(rate=0.3)  # randomly disable 30% of neurons

# 3. Early stopping
callback = tf.keras.callbacks.EarlyStopping(
    monitor='val_loss', patience=5, restore_best_weights=True
)
model.fit(X_train, y_train, callbacks=[callback], validation_split=0.2)
```

Topic 12 — Advanced Topics and Roadmap

Expert

Advanced Topics and Learning Roadmap

Transfer learning

Reuse a model pre-trained on millions of examples and fine-tune it on your small dataset. Reduces training from weeks to hours.

```
from tensorflow.keras.applications import VGG16

base_model = VGG16(weights='imagenet', include_top=False, input_shape=(224,224,3))
base_model.trainable = False   # freeze pre-trained weights

model = tf.keras.Sequential([
    base_model,
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(5, activation='softmax'),  # your 5 classes
])
# Fine-tune in hours instead of training from scratch in weeks
```

Learning roadmap

Stage	Tools and topics	Time investment
1 — Foundations	Python, NumPy, Pandas, Matplotlib	4-6 weeks
2 — Classical ML	scikit-learn: regression, classification, clustering, evaluation	6-8 weeks
3 — Deep Learning	TensorFlow or PyTorch, CNNs, RNNs, Transformers	8-12 weeks
4 — Practice	Kaggle competitions, real datasets, personal projects	Ongoing
5 — Advanced	Hugging Face, LLMs, MLOps, cloud deployment (SageMaker)	3-6 months

Quick reference — all 12 topics

Topic	One-line definition
What is ML	Machine learns rules from examples, not programmed explicitly
Supervised	Learn from labelled input-output pairs
Unsupervised	Find hidden patterns in unlabelled data
Regression	Predict a continuous number — price, temperature, salary
Classification	Predict a discrete category — spam, cat vs dog
Decision trees	Learn if-then rules from data. Random forest + XGBoost for power
Clustering	Group similar data points — K-Means, DBSCAN
Neural networks	Layered nodes that learn by adjusting weights via backpropagation
Deep learning	Many-layer networks for images, text, audio — CNN, LSTM, Transformer
Evaluation	Precision, recall, F1, AUC — choose metric for your problem
Overfitting	Model memorises training data — fix with regularisation and dropout
Advanced	Transfer learning, RL, Transformers, MLOps in production